Algorithms for Massive Data

Master in Data Science for Economics

Market-Basket Analysis

Beatrice Clavarino

beatrice.clavarino@studenti.unimi.it

March 2026

I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work, and including any code produced using generative AI systems. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me/us or any other person for assessment on this or any other course of study.

Contents

1 Dataset **3**

2 Data Organisation **3**
2.1 Computing Framework . . . 3
2.2 Data Discovery . . . 3

3 Preprocessing **3**
3.1 Dimensionality Reduction . . . 4
3.2 Unique Basket Identifiers . . . 4
3.3 Text Normalisation . . . 4
3.4 Basket Transformation and Integrity Validation . . . 4

4 Algorithms **4**
4.1 Support Threshold . . . 4
4.2 A-Priori Algorithm . . . 5
4.2.1 Overview . . . 5
4.2.2 Pass 1 — Frequent Singletons (L_1) . . . 5
4.2.3 Pass 2 — Frequent Pairs (L_2) . . . 5
4.2.4 Association Rules . . . 5
4.3 SON Algorithm . . . 5
4.3.1 Overview . . . 5
4.3.2 Pass 1 — Local Candidate Generation . . . 6
4.3.3 Pass 2 — Global Validation . . . 6
4.3.4 Association Rules . . . 6
4.3.5 Comparison with A-Priori . . . 6

5 Scalability **6**
5.1 A-Priori . . . 7
5.2 SON . . . 7

6 Experiments **7**
6.1 Experimental Setup . . . 7
6.2 Support Threshold Selection . . . 8
6.3 Number of SON Partitions . . . 8
6.4 Dataset Replication for Scalability . . . 8

7 Results and Discussion **8**
7.1 Frequent Itemsets . . . 8
7.2 Association Rules . . . 9
7.3 Algorithm Correctness . . . 10
7.4 Summary . . . 10

1 Dataset

This project implements a system for discovering frequent itemsets through Market-Basket Analysis, applied to the *IMDB Top 1000 Movies* dataset, publicly available on Kaggle. The dataset contains 1 000 records, each describing a highly rated film, with 16 attributes including title, release year, genre, rating, director, and the names of up to four lead actors.

Following the market-basket model, each movie is treated as a **basket**, and the actors listed under the `Star1`–`Star4` fields are treated as **items**. The goal is to identify sets of actors that frequently appear together across movies — that is, itemsets whose support meets or exceeds a minimum threshold s.

Of the 16 available columns, only 5 are relevant to this analysis: `Series_Title` (the basket identifier) and `Star1`, `Star2`, `Star3`, `Star4` (the items). The remaining 11 columns are discarded during preprocessing.

The algorithms implemented are designed to scale to datasets exceeding available main memory. Although the dataset is small enough to fit in memory, both A-Priori and SON are structured to minimise main-memory usage: A-Priori by counting only pairs of frequent items, and SON by processing data in partitions and making two passes over the full dataset.

2 Data Organisation

2.1 Computing Framework

Apache Spark is used as the distributed computing framework. The session is configured with 64 shuffle partitions to ensure efficient parallelism and to support the scaling experiments.

2.2 Data Discovery

An initial exploration of the raw dataset was conducted to map its structure onto the components of the market-basket model and to identify any data quality issues before preprocessing.

Two checks were performed on the `Series_Title` column, which serves as the natural basket identifier. First, a missing-value audit confirmed that no titles are null or empty. Second, a uniqueness check — applied after normalising titles to lowercase — revealed 999 unique titles out of 1 000 rows, indicating a duplicate.

Inspecting the duplicate reveals two distinct films both titled *Drishyam* (released in 2013 and 2015 respectively), with entirely different casts. Using the title as a primary key would therefore merge two different baskets into one, corrupting the support counts. This issue is resolved in the preprocessing phase by assigning a synthetic unique identifier to each row.

3 Preprocessing

Preprocessing serves two purposes: reducing the data footprint to reserve main memory for the computationally intensive counting steps, and ensuring the correctness of item-

occurrence counts.

3.1 Dimensionality Reduction

The dataset is restricted to the five relevant columns, eliminating 11 of the original 16. This is especially important because the main-memory bottleneck in frequent itemset mining arises primarily from counting candidate pairs.

3.2 Unique Basket Identifiers

A unique `Basket_ID` is generated for each row. This guarantees that each movie is treated as a distinct basket regardless of title, resolving the homonymy collision identified during data discovery.

3.3 Text Normalisation

To ensure accurate support counts, actor names are standardised through:

- **Case folding** (conversion to lowercase) to eliminate case inconsistencies that would produce false negatives (e.g., *"Al Pacino"* vs. *"al pacino"*).
- **Trimming** (removal of leading/trailing whitespace) to handle formatting artefacts in the source data.
- **Null imputation**: missing values in actor fields are replaced with empty strings, to be removed in the subsequent step.

3.4 Basket Transformation and Integrity Validation

Each row is transformed into a transactional format: the four actor columns are aggregated into a single array, from which empty strings are removed. Two additional constraints enforce the formal definition of a basket:

- **Deduplication**: each actor may appear at most once per basket, preventing inflated pair counts within a single transaction.
- **Empty basket removal**: movies with no listed actors are discarded, as they contribute no information to the analysis.

After preprocessing, the dataset consists of 1 000 baskets, each containing between 1 and 4 normalised actor names.

4 Algorithms

4.1 Support Threshold

The **support** of an itemset I, denoted $\sup(I)$, is the number of baskets that contain every item in I. An itemset is **frequent** if $\sup(I) \geq s$.

To ensure that results remain meaningful as the dataset grows, the threshold is defined as a fixed fraction of the total number of baskets N:

$$s = N \times \mathit{support_fraction}. \tag{1}$$

This design means that s scales automatically with data size, so no manual recalibration is needed when the dataset is replicated for the scalability experiments.

4.2 A-Priori Algorithm

4.2.1 Overview

The A-Priori algorithm discovers frequent itemsets by exploiting the **Monotonicity Property**: if an itemset is frequent, every subset of it must also be frequent. Equivalently, if an itemset is infrequent, all of its supersets are infrequent and need not be counted. This property allows the algorithm to prune the candidate space drastically at each level, avoiding the need to enumerate all possible itemsets.

In this implementation, A-Priori performs two passes over the data and discovers frequent itemsets up to size 2. The two-level design directly reflects the memory-awareness goal: by restricting the second pass to pairs of already-frequent singletons, the number of counters maintained in main memory is bounded by $\binom{|L_1|}{2}$ rather than $\binom{n}{2}$ for all n actors.

4.2.2 Pass 1 — Frequent Singletons (L_1)

Each actor is counted individually across all baskets. Only those whose count meets or exceeds s are retained as frequent singletons, forming the set L_1.

4.2.3 Pass 2 — Frequent Pairs (L_2)

Candidate pairs are generated exclusively from items in L_1. For each basket, actors not in L_1 are discarded, and all 2-combinations of the remaining frequent actors are enumerated and counted. Only pairs whose count reaches s are retained.

4.2.4 Association Rules

From L_2, association rules of the form $A \to B$ are extracted, where A and B are single actors. Each rule is evaluated using two metrics:

$$\mathrm{Conf}(A \to B) = \frac{\sup(A \cup B)}{\sup(A)}, \tag{2}$$

$$\mathrm{Interest}(A \to B) = \mathrm{Conf}(A \to B) - \Pr[B], \qquad \Pr[B] = \frac{\sup(B)}{N}. \tag{3}$$

Confidence measures the conditional probability that B appears given A. Interest corrects for the baseline frequency of B: a value close to 0 indicates that the co-occurrence is explained by B's individual popularity, whereas a value close to 1 indicates a strong genuine association between the two actors independently of their individual frequencies.

4.3 SON Algorithm

4.3.1 Overview

The SON algorithm (Savasere, Omiecinski, and Navathe) is designed for datasets that exceed main-memory capacity. It exploits the following theoretical guarantee:

An itemset can only be globally frequent if it is locally frequent in at least one partition of the dataset.

This ensures **no false negatives**: the union of all locally frequent itemsets is guaranteed to be a superset of all globally frequent ones. False positives (itemsets frequent locally but not globally) are then eliminated in a second pass over the full data.

4.3.2 Pass 1 — Local Candidate Generation

The dataset is divided into k partitions. Each partition represents a fraction $p = 1/k$ of the data, so its local support threshold is $p \cdot s$. Within each partition, an iterative A-Priori procedure is applied, extending beyond pairs to itemsets of arbitrary size by repeatedly applying the Monotonicity Property: a candidate of size k is only generated if all of its $(k-1)$-subsets are locally frequent. The candidates from all partitions are unified into the global candidate set $\mathcal{C}$.

4.3.3 Pass 2 — Global Validation

Every candidate in $\mathcal{C}$ is counted against the full dataset. Those whose global support falls below s are discarded as false positives.

4.3.4 Association Rules

Because SON discovers itemsets of arbitrary size, rules of the form $I \to j$ are extracted, where I is a set of one or more actors (antecedent) and j is a single actor (consequent). The same metrics apply:

$$\mathrm{Conf}(I \to j) = \frac{\sup(I \cup \{j\})}{\sup(I)}, \qquad \mathrm{Interest}(I \to j) = \mathrm{Conf}(I \to j) - \Pr[j]. \tag{4}$$

4.3.5 Comparison with A-Priori

The pairwise frequent itemsets (L_1 and L_2) produced by both algorithms are identical, validating the correctness of both implementations. SON additionally identifies frequent triples (L_3), from which richer hierarchical rules are derived — rules that A-Priori, in this two-pass implementation, cannot produce.

5 Scalability

To evaluate scalability, the preprocessed dataset was artificially replicated at 1×, 10×, and 100× its original size, simulating datasets of 1 000, 10 000, and 100 000 baskets respectively. Because the support threshold is defined as a fixed fraction of N (Equation 1), s scales proportionally with data size and no parameter re-tuning is required.

5.1 A-Priori

Table 1: A-Priori scalability results (`support_fraction` = 0.003)

Replication	**Baskets (N)**	s	**Time (s)**	$\lvert L_2 \rvert$
1×	1 000	3	0.67	25
10×	10 000	30	3.46	25
100×	100 000	300	52.66	25

The number of frequent itemsets remains constant across all replication factors, as expected: data replication does not alter relative item frequencies, and s scales proportionally so the same itemsets remain above threshold. Execution time grows approximately sub-linearly: a 100× increase in data yields roughly an 85× increase in time, consistent with the linear scan required by A-Priori.

5.2 SON

Table 2: SON scalability results (`support_fraction` = 0.003, $k = 2$ partitions)

Replication	**Baskets (N)**	s	$\lvert \mathcal{C} \rvert$	**Time (s)**	**Total itemsets**
1×	1 000	3	550	1.41	300
10×	10 000	30	300	6.44	300
100×	100 000	300	300	65.68	300

The number of frequent itemsets is stable across all replication factors. The candidate set $|\mathcal{C}|$ decreases from 550 to 300 after the first replication: a larger N raises the local threshold $p \cdot s$, causing fewer itemsets to qualify locally and thus reducing the number of false positives entering Pass 2. This confirms the theoretical trade-off between partition size and candidate quality.

SON is consistently slower than A-Priori due to the overhead of the two-pass MapReduce structure. However, this cost is justified by the ability to detect itemsets of size $k > 2$ and by its suitability for distributed settings where data does not fit in main memory. Both algorithms exhibit approximately linear growth in execution time with dataset size.

6 Experiments

6.1 Experimental Setup

All experiments were conducted on Google Colab using a single-node Apache Spark instance with 64 shuffle partitions. The dataset was loaded from the Kaggle IMDB Top 1000 Movies CSV file (175 KB, 1 000 rows after preprocessing). The SON algorithm used $k = 2$ partitions throughout, a natural choice for a dataset of this size and a sufficient baseline for assessing the two-pass overhead.

6.2 Support Threshold Selection

The support threshold is the most critical experimental parameter, as it directly controls the number and quality of the discovered frequent itemsets.

An initial threshold of `support_fraction` $= 0.005$ ($s = 5$) was evaluated first. At this level, A-Priori identified 78 frequent singletons but only 3 frequent pairs. The high sparsity of the dataset — each basket contains at most 4 items, and film casts rarely overlap with more than a handful of others — makes a 0.5% threshold too restrictive to yield informative results.

The threshold was subsequently lowered to `support_fraction` $= 0.003$ ($s = 3$). At this level, L_1 grows to 272 frequent actors and L_2 to 25 frequent pairs, producing a rich and interpretable set of association rules while remaining computationally tractable. This threshold was adopted for all subsequent experiments.

6.3 Number of SON Partitions

The number of partitions k governs the local threshold $p \cdot s = s/k$, and therefore the volume of false positives entering Pass 2. With $k = 2$ and $s = 3$, the local threshold is 1.5: any itemset appearing at least twice in a partition qualifies as a local candidate. Increasing k would lower the local threshold, generating more candidates at the cost of more false positives to filter; decreasing k would raise the local threshold, reducing candidates but risking missed globally frequent itemsets. For the dataset size used here, $k = 2$ represents a conservative and effective choice.

6.4 Dataset Replication for Scalability

To stress-test both algorithms beyond 1 000 baskets, the preprocessed dataset was replicated 10× and 100× by taking repeated unions of the original Spark DataFrame. This approach preserves the exact frequency distribution, ensuring that the number of frequent itemsets remains constant while execution time scales with volume. This design isolates the computational cost of each algorithm from any change in result structure, providing a clean measure of scalability.

7 Results and Discussion

7.1 Frequent Itemsets

With `support_fraction` $= 0.003$ ($s = 3$), both algorithms agree on 272 frequent singletons (L_1) and 25 frequent pairs (L_2). SON additionally identifies 3 frequent triples (L_3). The top frequent itemsets by size are shown in Table 3.

Table 3: Top frequent itemsets by size (SON, $s = 3$)

Level	Itemset	Support
L_1	robert de niro	17
	tom hanks	14
L_2	{daniel radcliffe, rupert grint}	6
	{daniel radcliffe, emma watson}	5
L_3	{daniel radcliffe, emma watson, rupert grint}	5
	{elijah wood, ian mckellen, orlando bloom}	3
	{carrie fisher, harrison ford, mark hamill}	3

The three frequent triples correspond directly to well-known film franchises: the *Harry Potter* series, *The Lord of the Rings* trilogy, and the original *Star Wars* trilogy. This is a natural outcome of the dataset structure: franchise films featuring the same recurring cast account for multiple baskets, making co-occurrence of lead actors near-deterministic.

7.2 Association Rules

Table 4 reports the top 20 association rules by Interest, as produced by A-Priori. The SON algorithm generates the same pairwise rules, plus additional hierarchical rules derived from L_3.

Table 4: Top 20 association rules by Interest (A-Priori, $s = 3$)

Antecedent	Consequent	Conf.	Interest
elijah wood	orlando bloom	1.000	0.997
orlando bloom	elijah wood	1.000	0.997
mark hamill	carrie fisher	1.000	0.996
daniel radcliffe	rupert grint	1.000	0.994
rupert grint	daniel radcliffe	1.000	0.994
john turturro	ethan coen	1.000	0.994
elijah wood	ian mckellen	1.000	0.993
orlando bloom	ian mckellen	1.000	0.993
mark hamill	harrison ford	1.000	0.992
tim allen	tom hanks	1.000	0.986
daniel radcliffe	emma watson	0.833	0.826
rupert grint	emma watson	0.833	0.826
carrie fisher	mark hamill	0.750	0.747
joe russo	chris evans	0.750	0.744
takashi shimura	toshirô mifune	0.750	0.743
joe russo	robert downey jr.	0.750	0.743
carrie fisher	harrison ford	0.750	0.742
lauren bacall	humphrey bogart	0.750	0.741
emma watson	daniel radcliffe	0.714	0.708
emma watson	rupert grint	0.714	0.708

All top-20 rules exhibit high Confidence and Interest. The Interest metric is particularly

informative: values close to 1.0 confirm that the consequent actor has a low baseline probability $\Pr[B]$, yet appears almost exclusively alongside the antecedent. This rules out the possibility that the association is a spurious artefact of one actor's overall popularity, and confirms a genuine structural co-occurrence driven by shared franchise membership.

The SON algorithm additionally produces hierarchical rules such as:

$$[\text{Elijah Wood}] \rightarrow [\text{Ian McKellen, Orlando Bloom}] \quad \text{Conf} = 1.0,$$

$$[\text{Carrie Fisher, Harrison Ford}] \rightarrow [\text{Mark Hamill}] \quad \text{Conf} = 1.0.$$

These rules capture the full three-actor co-occurrence structure of franchise casts — something pairwise rules alone cannot express — and represent the primary added value of SON over the two-pass A-Priori implementation.

7.3 Algorithm Correctness

The consistency of L_1 and L_2 across both algorithms provides a strong mutual validation of the two implementations. Any discrepancy would have indicated a bug in SON's global candidate generation or counting phase, or in A-Priori's pair-generation logic.

7.4 Summary

Market-Basket Analysis proved effective for discovering latent collaboration structures in film data. The use of a fractional support threshold was essential for obtaining meaningful results on a sparse dataset, and the combined use of Confidence and Interest allowed genuine associations to be distinguished from coincidental co-occurrences. Both algorithms scale approximately linearly with data size, validating their design as memory-conscious, scalable solutions suited for deployment on larger real-world datasets.